The Judger already writes as fast as this GPU allows

Hybrid LatentMAS serving · Qwen3-14B · one H200 · greedy

Sagnik Biswas

13 October 2026

What this round was for

The earlier result still stands: on a long answer, the three silent agents are about a second, and the Judger's written answer is the rest of the clock.

This round asked one question. **Can the serving stack make that clock shorter**, with the model, the prompts, the latent-step count, and the decoding policy held fixed?

- Starting point: the authors' optional fast path. Planner, Critic, and Refiner stay in Hugging Face. The Judger runs in vLLM.
- Hardware: one H200. Stack: PyTorch 2.8, Transformers 4.57.1, vLLM 0.11.0.
- The authors' published numbers are the Hugging Face-only path. This hybrid path is a separate implementation. It is not claimed to match those numbers bit for bit.

Where one request spends its time

Repaired hybrid path, one question, 1024 forced tokens, greedy.

piece	seconds	share
three latent agents, 10 steps each	1.07	10%
hand the vectors to vLLM	$\sim$0	—
Judger writes 1024 tokens	9.37	89%
end to end	10.49	100%

- The latent shortcut is doing its job. Those agents never sample a word. They feed the hidden vector back in as the next input.
- The Judger cannot skip that. A person needs the written answer.
- Moving the embedding tensor onto the GPU is not the cost.

The authors' hybrid code has two real bugs

These are correctness bugs. Fixing them does not make a request faster. The repaired path is what every timing below uses. The original path is still the default, so the crash stays reproducible.

- **Batch size 1 crashes.** The code squeezes a 3D embedding down to 2D, then tries to unpack three dimensions. `ValueError: not enough values to unpack (expected 3, got 2)`. Recorded after a full model load.
- **A batched call is not the same as separate calls.** Padding positions are kept, and the pad value is float32, so a bf16 embedding is promoted. The repaired path drops masked positions and keeps bf16.

Giving vLLM vectors does not slow the writing

vLLM normally wants token ids. The hybrid Judger has to hand it embedding rows instead. vLLM's own comments say that mode leaves the embedding lookup outside the compiled graph, which sounded like a tax on every generated token.

Same engine, same GPU, 1024 greedy tokens, no latent agents in either run.

how the engine was built	seconds	tokens/s
ordinary text prompt	9.37	109.3
prompt embeddings enabled	9.51	107.7

A raw memory copy on this H200 hits 4.26 TB/s. A 14B bf16 weight read then caps batch-1 decode near 152 tokens/s. **109 is ordinary batch-1 decode**, about 72% of that ceiling.

Hiding the next latent rollout behind the Judger does not pay

On one GPU the two pieces contend. Four requests, 1024 forced tokens.

arm	wall clock
one request fully finishes, then the next starts	41.52 s
build request $i{+}1$ while vLLM writes request i	40.75 s

- Speedup $\mathbf{1.019}\times$.
- Decode slowed from about 9.6 s to about 10.1 s whenever the next latent rollout ran on the same card. That cancels the $\sim$0.8 s of prep.
- The greedy text matched across arms, so the overlap did not corrupt the answer. This pod had one GPU. The authors' two-GPU split was not measured.

Prefix caching does not engage

vLLM 0.11 turns prefix caching off when prompt embeddings are enabled. We checked the live engine, not a timing guess.

- `enable_prefix_caching` on the running engine: **false**.
- Replaying the identical embedding, and a copy with the middle row shifted by 1: `num_cached_tokens` $= 0$ both times (0.097 s vs 0.095 s).
- The first eight tokens matched because both answers open with `<think>`. That is the model, not a cache hit.

On real questions the number is right, and the essay is the cost

Three saved questions. Greedy, stop on EOS, cap 512. None hit the cap.

question	tokens	end to end	of which vLLM	answer
add	257	3.48 s	2.40 s	42
packs	293	3.56 s	2.73 s	30
long	499	5.82 s	4.86 s	37

- All three matched the gold integer.
- Each answer is a long `<think>` trace that ends in `\boxed{42}` (or 30, or 37).
- Latent rollout stays about 0.8–1.0 s. Once the answer is a few hundred tokens, vLLM is most of the request.

Batching is the throughput lever that already works

Fixed 1024 tokens, so a shorter answer cannot fake a speedup. One wall-clock number per batch.

batch	wall	latent	vLLM
1	10.49 s	1.07 s	9.37 s
2	10.36 s	0.85 s	9.50 s
4	10.54 s	0.89 s	9.63 s
8	13.00 s	2.25 s	10.68 s

Eight requests finish in 13.0 s. Eight serial requests would be about 84 s. That is about **6.5**$\times$ the single-request rate, from the offline batch the code already had. Zero errors across this sweep.

What to take from this

We did not make a single request faster than the hybrid path we started from.

- The Judger writes at ∼109 tokens/s. A plain text prompt on the same vLLM build writes at the same speed.
- Previously, the Hugging Face Judger on this model measured ∼42.5 tokens/s. vLLM was already that gain. It is a different workload, so it is context, not a paired A/B from this week.
- The repair is worth keeping: batch size 1 no longer crashes, and a batch matches the requests that make it up.
- The open lever is how many words the Judger writes. Same gold number, fewer tokens, and the wall clock falls with them. A shorter wrong answer does not count.